TerraFormars

IaC AI Platform

Especificación de Requisitos de Software (SRS)

Estándar IEEE 830 / ISO/IEC/IEEE 29148

Proyecto de Grado — Fundación Universitaria Católica Lumen Gentium

Institución:	Fundación Universitaria Católica Lumen Gentium
Modalidad:	Proyecto de Grado
Versión:	1.1 (Security Hardened)
Fecha:	6 de Agosto de 2026

Documento generado conforme a IEEE 830 / ISO/IEC/IEEE 29148

Tabla de Contenido

1. Introducción

2. Descripción General

3. Requisitos Funcionales

4. Requisitos No Funcionales

5. Requisitos de Interfaz Externa

6. Matriz de Trazabilidad

1. Introducción

1.1 Propósito

El propósito de este documento es definir la Especificación de Requisitos de Software (SRS) para la plataforma **TerraFormars**. Este documento detalla los requisitos funcionales, no funcionales, interfaces del sistema y restricciones de diseño para orientar el desarrollo, pruebas, mantenimiento y despliegue del sistema.

1.2 Alcance

TerraFormars es una solución Software as a Service (SaaS) diseñada para automatizar la creación, gestión, despliegue y monitoreo de infraestructura en **Microsoft Azure** utilizando **Inteligencia Artificial (OpenAI GPT-4o)** y **Terraform CLI**.

El sistema abarca:

- Conversión de requerimientos en lenguaje natural a código Terraform HCL funcional.
- Gestión segura de credenciales multi-tenant de Microsoft Azure (Service Principals).
- Ejecución asíncrona de despliegues mediante colas de tareas con logs en tiempo real vía WebSockets.
- Monitoreo de recursos en tiempo real mediante telemetría de Azure Monitor API.
- Ejecución remota de scripts (PowerShell/Bash) en Máquinas Virtuales de Azure.
- Administración y visualización unificada de recursos en Azure.

1.3 Personal Involucrado y Audiencia

Este documento está dirigido a:

- Autor(es) del proyecto de grado y equipo de desarrollo.
- Equipos de DevOps, SRE e Infraestructura Cloud.
- Evaluadores de Calidad y Pruebas (QA).
- Administradores de Sistemas y Seguridad.

1.4 Definiciones, Acrónimos y Abreviaturas

Término	Definición
`IaC`	Infrastructure as Code — Infraestructura como Código.
`HCL`	HashiCorp Configuration Language — lenguaje sintáctico usado por Terraform.
`Service Principal (SPN)`	Identidad creada en Azure Active Directory para uso con automatizaciones.
`AES-256`	Estándar de Cifrado Avanzado con clave de 256 bits.
`JWT`	JSON Web Token — estándar abierto para la creación de tokens de acceso firmados.

Término	Definición
`BullMQ`	Librería de procesamiento de colas y tareas asíncronas basada en Redis.
`AKS`	Azure Kubernetes Service.
`VNet`	Virtual Network — Red Virtual de Azure.

2. Descripción General

2.1 Perspectiva del Producto

TerraFormars es un sistema autónomo y multi-usuario que actúa como capa de abstracción entre los usuarios (ingenieros, DevOps, arquitectos) y los servicios de Microsoft Azure / OpenAI.

Capa	Componente	Descripción
Cliente	Frontend Next.js	Interfaz de usuario web responsiva (Mantine UI).
Servicio	Backend NestJS API	Orquesta autenticación, tenants, IA, despliegues y monitoreo.
Datos	PostgreSQL / Redis	Persistencia relacional y colas/cache asíncronas.
IA	OpenAI GPT-4o	Generación y refinamiento de código HCL, estimación de costos.
Infraestructura	Terraform CLI / Azure ARM	Ejecución de despliegues sobre suscripciones de Azure.

[Usuario Web] → [Frontend Next.js] → [Backend NestJS API] → { PostgreSQL/Redis · OpenAI GPT-4o · Terraform CLI/Azure ARM }

2.2 Funciones del Producto

#	Función	Descripción
1	**Generación IA de Plantillas IaC**	Creación y refinamiento iterativo de código HCL para Azure.
2	**Estimación y Escalado Dinámico**	Cálculo de costos estimados y detección de parámetros de rendimiento configurables.
3	**Gestión de Tenants Azure**	Almacenamiento cifrado de credenciales de múltiples suscripciones de Azure.
4	**Despliegue Asíncrono de Infraestructura**	Encolamiento y ejecución aislada de Terraform con transmisión de logs en vivo.
5	**Inventariado y Trazabilidad**	Registro automático del inventario de recursos generados y limpieza automática.
6	**Monitoreo en Tiempo Real**	Telemetría en vivo (CPU, RAM, Disco, Red) para VMs y recursos desplegados.
7	**Consola y Ejecución de Scripts**	Catálogo y ejecución remota de automatizaciones PowerShell/Bash sobre VMs.
8	**Administración Directa de Recursos Azure**	Inspección de VMs, AKS, App Services, VNets y gestión de VNet Peerings.

2.3 Características de los Usuarios

Perfil	Descripción
Administrador de Cloud / DevOps	Conocimiento avanzado de nube; busca acelerar despliegues y automatizar tareas repetitivas.
Desarrollador / Ingeniero de Software	Requiere provisionar ambientes de desarrollo/pruebas rápidamente sin escribir HCL complejo.
Arquitecto de Infraestructura	Diseña plantillas, valida normas de seguridad y evalúa costos estimados.

2.4 Restricciones del Sistema

• Nube Objetivo: exclusivamente Microsoft Azure (proveedor **azurerm** v4.x).

• Entorno de Ejecución Backend: requiere la instalación del ejecutable oficial **terraform** CLI en el contenedor/host del servidor backend.

• Dependencias de Terceros: requiere conectividad continua con la API de OpenAI y las APIs REST de Azure ARM / Monitor.

• Aislamiento Local: prohibición del uso de módulos locales de Terraform (**module "./..."**), forzando plantillas planas en un único archivo raíz (**main.tf**).

3. Requisitos Funcionales

Cada requisito funcional (RF) se identifica de forma unívoca y se agrupa por módulo, conforme a las buenas prácticas de trazabilidad de ISO/IEC/IEEE 29148.

3.1 Módulo de Autenticación y Usuarios (RF-01)

ID	Descripción del Requisito
`RF-01.1`	**Registro e Inicio de Sesión:** El sistema debe permitir a los usuarios registrarse e iniciar sesión utilizando correo electrónico y contraseña.
`RF-01.2`	**Cifrado de Contraseñas:** Las contraseñas deben ser hasheadas usando `bcrypt` antes de ser almacenadas en la base de datos.
`RF-01.3`	**Sesiones Stateless:** La autenticación debe manejarse mediante tokens JWT devueltos al cliente y validados en cada petición mediante `JwtAuthGuard`.

3.2 Módulo de Tenants de Azure (RF-02)

ID	Descripción del Requisito
`RF-02.1`	**Conexión de Tenants:** El sistema debe permitir guardar credenciales de Azure (tenant_id, client_id, subscription_id y client_secret).
`RF-02.2`	**Cifrado de Secretos:** El client_secret debe cifrarse mediante el algoritmo **AES-256** antes de su persistencia en PostgreSQL.
`RF-02.3`	**Protección de Secretos:** Las respuestas API de consulta de tenants no deben incluir el client_secret cifrado ni en texto plano.

3.3 Módulo de Generación por IA (RF-03)

ID	Descripción del Requisito
`RF-03.1`	**Generación de Código HCL:** El sistema debe recibir una descripción sintáctica en lenguaje natural y retornar código Terraform HCL estructurado, sin bloques de código en formato markdown.
`RF-03.2`	**Reglas Estrictas de Generación:** La IA debe aplicar reglas sintácticas obligatorias (p. ej. computer_name sin guiones bajos, validación de SKUs válidos para VMs Windows/Linux, restricciones de nombres para Storage Accounts, etc.).
`RF-03.3`	**Detección de Variables:** El sistema debe extraer automáticamente las variables definidas en el código HCL (nombre, tipo, descripción, valor por defecto).
`RF-03.4`	**Detección de Atributos Escalables:** La IA debe analizar el código HCL y retornar un objeto JSON con los atributos de escala detectados (p. ej. vm_size, sku_name) ofreciendo hasta 10 opciones válidas de Azure.
`RF-03.5`	**Estimación de Costos:** El sistema debe retornar un valor estimado mensual en USD calculado mediante GPT-4o según los recursos especificados.

ID	Descripción del Requisito
RF-03.6	**Refinamiento Iterativo:** El sistema debe permitir editar un código HCL existente mediante un nuevo prompt sin perder la estructura previa.

3.4 Módulo de Despliegues (RF-04)

ID	Descripción del Requisito
RF-04.1	**Creación de Despliegue:** El sistema debe permitir crear una orden de despliegue asociando una plantilla, un tenant de Azure y los valores para los parámetros/variables.
RF-04.2	**Procesamiento Asíncrono:** Los despliegues deben encolarse en una cola de Redis manejada por BullMQ (cola deployments).
RF-04.3	**Aislamiento de Entorno:** Cada ejecución de Terraform debe llevarse a cabo en un directorio temporal aislado (/tmp/terraform-{timestamp}).
RF-04.4	**Streaming de Logs:** El sistema debe emitir los logs de salida de terraform init y terraform apply en tiempo real vía WebSockets al cliente frontend.
RF-04.5	**Registro de Inventario:** Tras un despliegue exitoso, el sistema debe registrar el JSON de recursos creados (resources_inventory).
RF-04.6	**Eliminación con Limpieza Azure:** Al eliminar un despliegue registrado, el sistema debe disparar la eliminación en segundo plano del Grupo de Recursos asociado en Azure.
RF-04.7	**Diagnóstico Inteligente de Errores:** En caso de falla (FAILED), el sistema debe analizar los patrones de error en los logs y retornar consejos explicativos al usuario.

3.5 Módulo de Monitoreo y Telemetría (RF-05)

ID	Descripción del Requisito
RF-05.1	**Polling de Telemetría:** El sistema debe consultar la API de Azure Monitor cada 15 segundos para recursos activos de tipo VM.
RF-05.2	**Métricas de Cómputo:** Debe recopilar: porcentaje de CPU, Network In/Out, Disk Read/Write Bytes y memoria disponible.
RF-05.3	**Transmisión WebSocket:** El gateway de monitoreo (MonitoringGateway) debe transmitir las métricas recopiladas a los clientes suscritos al resourceId.
RF-05.4	**Ciclo de Vida de Suscripción:** Al desconectarse el último cliente de un recurso, el temporizador de polling debe detenerse para liberar memoria.

3.6 Módulo de Scripts y Ejecución Remota (RF-06)

ID	Descripción del Requisito
RF-06.1	**Catálogo de Scripts:** Permite guardar, categorizar y editar scripts en lenguaje Bash o PowerShell.
RF-06.2	**Generación IA de Scripts:** Genera scripts en nativo para el SO objetivo, evitando el uso innecesario de az cli.
RF-06.3	**Ejecución Ad-Hoc o Asociada:** Ejecuta scripts sobre recursos tipo VM utilizando la API Run Command de Azure (RunPowerShellScript para Windows, RunShellScript para Linux).
RF-06.4	**Cancelación de Ejecución:** Permite cancelar ejecuciones en estado PENDING o RUNNING removiendo el trabajo de la cola BullMQ.

3.7 Módulo de Administración de Azure (RF-07)

ID	Descripción del Requisito
RF-07.1	**Exploración Unificada:** Visualización de recursos existentes en el tenant seleccionado: VMs, AKS Clusters, App Services, VNets y bases de datos PostgreSQL.
RF-07.2	**Gestión de VNet Peering:** Permite crear y eliminar interconexiones bidireccionales (VirtualNetworkPeering) entre dos redes virtuales.
RF-07.3	**Inspección de AKS:** Consulta de pods, nodos y servicios de un clúster de Kubernetes leyendo el kubeconfig de administrador obtenido vía Azure SDK.

4. Requisitos No Funcionales

4.1 Seguridad (RNF-01)

Atributo	Especificación
Cifrado en Reposo	Claves secretas de Azure cifradas con AES-256.
Cifrado en Tránsito	Toda la comunicación entre cliente y servidor debe soportar HTTPS y WSS (WebSockets Secure).
Aislamiento Multi-tenant	Los usuarios sólo pueden visualizar y operar sobre sus propios datos e instalaciones.
Sanitización de Salidas	Se prohíbe el volcado de contraseñas o secretos en los logs de la aplicación.

4.2 Rendimiento y Escalabilidad (RNF-02)

Atributo	Especificación
Procesamiento No Bloqueante	El servidor backend no debe bloquear el hilo principal de Node.js durante los despliegues de Terraform; toda tarea pesada se delega a workers de BullMQ.
Carga Única de Polling	Múltiples usuarios monitoreando el mismo recurso en la UI deben reutilizar el mismo timer de polling en el backend.
Tiempos de Respuesta de API	Endpoints REST sin llamadas a APIs externas deben responder en menos de 300 ms.

4.3 Disponibilidad y Mantenibilidad (RNF-03)

Atributo	Especificación
Arquitectura en Contenedores	Toda la aplicación debe desplegarse mediante Docker Compose con reinicio automático (restart: always).
Mantenibilidad	El código backend debe seguir la estructura modular de NestJS (Inyección de dependencias, controladores, servicios y entidades TypeORM).

4.4 Internacionalización (RNF-04)

Atributo	Especificación
Soporte Multi-idioma	La interfaz del frontend debe soportar los idiomas Español (ES) e Inglés (EN) en su totalidad mediante react-i18next.

5. Requisitos de Interfaz Externa

5.1 Interfaz de Usuario (UI)

- Desarrollada en **Next.js** con la librería **Mantine UI**.
- Diseño responsive adaptado a pantallas de escritorio y tablets.
- Soporte para Tema Oscuro (Dark Mode).

5.2 Interfaces de Software

Componente	Tecnología / SDK
Microsoft Azure SDK	Uso de SDKs oficiales @azure/arm-* y @azure/identity.
Terraform CLI	Ejecución binaria nativa local mediante llamadas a subprocesos (child_process).
OpenAI API	Modelo gpt-4o mediante la SDK oficial openai.
Base de Datos	PostgreSQL 15 con ORM TypeORM.
Cache / Colas	Redis 7 con la librería BullMQ.

5.3 Interfaces de Comunicación

Protocolo	Uso
REST API	Protocolo HTTP/HTTPS con formato JSON.
WebSockets	Protocolo WS/WSS mediante Socket.io para streaming de logs y telemetría.

6. Matriz de Trazabilidad

La siguiente matriz vincula cada requisito funcional con su módulo y las entidades/servicios responsables de su implementación, conforme a los lineamientos de trazabilidad de ISO/IEC/IEEE 29148.

ID Requisito	Módulo	Entidad / Servicio
RF-01	Autenticación	`User, AuthService, JwtStrategy`
RF-02	Tenants	`AzureTenant, TenantsService, EncryptionService`
RF-03	Generador IA	`OpenAIService, Template`
RF-04	Despliegues	`Deployment, DeploymentsService, DeploymentProcessor, TerraformService`
RF-05	Monitoreo	`MonitoringService, MonitoringGateway`
RF-06	Scripts	`Script, ScriptExecution, ScriptsService, ScriptExecutionProcessor`
RF-07	Admin / AKS	`AdminService, K8sService`

Documento SRS generado para TerraFormars — Proyecto de Grado, Fundación Universitaria Católica Lumen Gentium.